

Android（Java + XML）基礎入門 + AI 互動指南

環境建置與基礎：建立你的第一個 Android App

教學內容 + AI 提示互動指南（穿插版）

本講對象與路徑

- 對象：具備 Java 開發經驗，熟悉 JFrame 視窗程式開發
- 路徑：Java + XML 佈局 + 資料庫 / 檔案存取
- 時間：約 6–8 小時
- ⚡ 與 JFrame 的對照會用 ⚡ 標記

先備知識（你應該已具備）

- Java 基本語法（類別、方法、變數、迴圈、if/else）
- 繼承、介面、Listener 事件模型
- 建立 GUI 元件與佈局的觀念
- (加分) 基本 SQL 語法

怎麼用這份投影片

- **前半教學**：每個主題先講「做法」
- **穿插的「AI 動手做」**：緊接在該主題之後，教你「把 AI 當工具」產出同一段程式
- **心法**：你決定做什麼（需求），AI 寫「怎麼做」（程式碼），但你必須能「看懂、改、驗證」

第 1 章 安裝環境

1. 下載 Android Studio（官方 IDE）→ <https://developer.android.com/studio>
2. 安裝時勾選 Android Virtual Device (AVD) 建立模擬器

建立虛擬裝置 (Emulator)

- 頂部工具列 → Device Manager → Create device
- 選一台手機型號（建議 Pixel 系列）、下載 System Image（API 34 / Android 14）
- 完成後點  啟動模擬器

⚡ 模擬器就像本地端的 JVM，但它是完整的手機系統。

第 2 章 建立第一個專案

1. File → New → New Project
2. 選 **Empty Views Activity**（不是 Compose，因為用 XML）
3. Project name：MyFirstApp
4. Language 選 **Java**
5. Minimum SDK 選 API 24 以上

AI 動手做 | 環境建置需要 AI 嗎？（指南 §1）

不需要。 直接照第 1、2 章做即可：

1. 安裝 Android Studio + 建立模擬器（AVD）
2. File → New → New Project → **Empty Views Activity**，Language 選 Java

之後每個範例都回到「New Project」或「複製既有專案」建立，再覆蓋 XML 與 Java。

AI 的價值在「程式碼」，環境安裝靠自己動手。

第 3 章 Android 專案結構（對照 JFrame）

資料夾 / 檔案	用途	JFrame 對照
app/src/main/java/...	Java 原始碼	src/
app/src/main/res/layout/activity_main.xml	畫面佈局	setContentPane + 拉元件
app/src/main/res/values/strings.xml	字串資源	硬編碼字串
app/src/main/AndroidManifest.xml	宣告 Activity、權限	沒有直接對照
app/build.gradle	專案設定 / 依賴	pom.xml / build.gradle

最重要的一點：Android 的 UI 預設用 XML 描述，Java 程式只做邏輯與綁定。

第 4 章 第一個畫面：activity_main.xml

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">

    <TextView android:id="@+id/tvTitle"
        android:layout_width="wrap_content" android:layout_height="wrap_content"
        android:text="我的第一個 App" android:textSize="24sp" android:textStyle="bold" />

    <EditText android:id="@+id/etInput"
        android:layout_width="match_parent" android:layout_height="wrap_content"
        android:hint="請輸入文字" />

    <Button android:id="@+id/btnShow"
        android:layout_width="match_parent" android:layout_height="wrap_content"
        android:text="顯示輸入" />

    <TextView android:id="@+id/tvResult"
        android:layout_width="wrap_content" android:layout_height="wrap_content"
        android:text="結果：" android:textSize="18sp" />

</LinearLayout>
```

佈局說明（activity_main.xml）

根元素 `<LinearLayout>`（容器）

- `xmlns:android`：宣告 Android 命名空間
- `android:orientation="vertical"`：子元件由上到下堆疊
- `layout_width/height="match_parent"`：填滿父容器
- `padding="16dp"`：四邊留內距

四個子元件（由上看下）

1. `tvTitle`： `wrap_content`（寬度只包住文字）、24sp 加粗
2. `etInput`： `match_parent` 佔滿寬度； `hint` 是空框的灰色提示
3. `btnShow`： `match_parent`、顯示「顯示輸入」
4. `tvResult`： `wrap_content`、初始「結果：」

`@+id/xxx` 就是程式 `findViewById(R.id.xxx)` 抓的 id，名字必須對得上。

4.1 理解 LinearLayout

LinearLayout 就像 Swing 的 BorderLayout 。

- orientation="vertical" → 垂直排列； horizontal → 水平排列
- 兩種尺寸值：
 - wrap_content：跟著內容大小（≈ Swing preferred size）
 - match_parent：填滿父容器（≈ fill）

4.2 尺寸單位

單位	用途
dp	密度無關像素（排版用，推薦）
sp	縮放像素（ 文字 用，跟隨系統字型）
px	實際像素（通常不用）

⚡ Swing 用 setSize 像素；Android 用 dp，依螢幕密度自動縮放。文字請用 sp 尊重無障礙字型偏好。

AI 動手做 | 只產生 XML 佈局（指南 §3・提示 2）

學完佈局，想「只」要佈局檔、練習自己寫 Java？

請幫我寫一個 Android XML 佈局（activity_main.xml）：

- LinearLayout，垂直排列，padding 24dp
- 一個標題 TextView「我的第一個 App」24sp 粗體
- 一個 EditText（id=etInput），hint「請輸入文字」，match_parent
- 一個 Button（id=btnShow）「顯示輸入」，match_parent
- 一個 TextView（id=tvResult）「結果：」18sp

請用 XML code block 輸出，不用解說

反過來用 AI 學習：

請用 JFrame 的角度跟我解釋 LinearLayout 的 match_parent 和 wrap_content 各是什麼？

第 5 章 綁定元件與事件（MainActivity.java）

```
public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);    // 載入 XML 佈局

        TextView tvTitle = findViewById(R.id.tvTitle);
        EditText etInput = findViewById(R.id.etInput);
        Button btnShow = findViewById(R.id.btnShow);
        TextView tvResult = findViewById(R.id.tvResult);

        btnShow.setOnClickListener(v -> {
            String input = etInput.getText().toString();
            tvResult.setText("結果：" + input);
            Toast.makeText(this, "你輸入了：" + input, Toast.LENGTH_SHORT).show();
        });
    }
}
```

MainActivity 程式說明

- `extends AppCompatActivity`：繼承「相容性 Activity」基底類別（≈ 主視窗）
- `onCreate(Bundle savedInstanceState)`：Activity 建立時系統**自動呼叫**（≈ `main` 切入點）
- `super.onCreate(savedInstanceState)`：**必須**先呼叫父類別實作
- `setContentView(R.layout.activity_main)`：告訴系統「畫面用哪支 XML」
- 綁定元件：四個 `findViewById(R.id.xxx)` 依 id 拿參考
- 綁定事件：`btnShow.setOnClickListener(v -> { ... })`
 - `etInput.getText().toString()`：`getText()` 回傳 `Editable`，需 `toString()`
 - `Toast.makeText(context, msg, Toast.LENGTH_SHORT).show()`：短暫訊息

⚡ Swing 先 `new JButton()` 再 `add()`；Android 先宣告 XML，再用 `findViewById` 拿參考。

AI 動手做 | 產生第一個畫面（指南 §2・提示 1）

綁定 + lambda 都會了，一次請 AI 產出「XM + Java」整套：

我是 Java 開發者，熟悉 Swing/JFrame，正在用 Android Studio 學 Android（Empty Views Activity，Java，XML 佈局）。

請幫我產生「顯示輸入文字」的第一個 App：

1. activity_main.xml：一個 TextView 標題、一個 EditText、一個 Button、一個 TextView 結果
 2. MainActivity.java：用 findViewById 綁定，點擊按鈕後用 lambda 把 EditText 的文字顯示到結果 TextView，並用 Toast 提示
- 套件名用 com.example.myfirstapp
請用 lambda 寫法

這是整份教材的「正確用法」：你給需求 → AI 給程式 → 你照貼 → 跑 → 驗證。

5.2 事件處理：lambda 與傳統對照（寫法比較）

```
// ===== 傳統：anonymous class（類似 Swing） =====  
btnShow.setOnClickListener(new View.OnClickListener() {  
    @Override  
    public void onClick(View v) { /* 讀輸入、設結果 */ }  
});  
  
// ===== lambda（同一個東西） =====  
btnShow.setOnClickListener(v -> { /* 讀輸入、設結果 */ });  
  
// 只有一行動作時，連大括號都可省略  
btnShow.setOnClickListener(v -> tvResult.setText("已按下"));
```

- 兩者**完全等價**，行為一模一樣
- lambda 只是把「匿名類別 + 唯一方法」的樣板精簡掉

5.2 等價關係對照表

傳統 anonymous class	lambda
<code>new View.OnClickListener() { ... }</code>	<code>v -> { ... }</code>
參數型別 <code>(View v)</code>	型別可省略、自動推斷
方法體 <code>{ ... }</code>	箭頭 <code>-></code> 後區塊

結論：如果你會寫 Swing 的 `new ActionListener() { public void actionPerformed(...) {...} }`，那一整套樣板就是在 Android 用 `v -> { ... }` 取代。

5.2 進階：方法參考（method reference）

```
private void showInput() { ... }  
  
btnShow.setOnClickListener(v -> showInput()); // lambda  
btnShow.setOnClickListener(this::showInput); // 方法參考，更精簡  
// 兩者完全等價
```

條件提醒

- Lambda 需 Java 8+（Android Studio 預設支援，自動 desugaring）
- 只有 **單一抽象方法介面** 可用 lambda；多方法的介面仍需 anonymous class
- 方法覆寫（如 `onActivityResult`）**不能用** lambda

AI 動手做 | 專講 lambda vs anonymous（指南 §4・提示 3）

Android 的 `View.OnClickListener` 何時可以用 lambda？何時不能用？
請用 `JFrame` 的 `addActionListener` 對照，分別給我 lambda 和 `anonymous class` 兩種寫法。
順便說明 `onActivityResult`、`OnItemSelectedListener` 為何不能用 lambda。

這正是 `Appendix_B_Lambda.md` 的內容——用 AI 重問一遍當複習。

第 6 章 Toast 與 Log（取代 JOptionPane）

```
// Toast：螢幕下方短暫訊息（非同步、不阻塞）
Toast.makeText(this, "訊息內容", Toast.LENGTH_SHORT).show();
// LENGTH_SHORT ≈ 2 秒 / LENGTH_LONG ≈ 3.5 秒

// Log：寫到 Logcat 視窗（很像 System.out.println）
Log.d("TAG", "這是除錯訊息");
// 層級由低到高：v(verbose) → d(debug) → i(info) → w(warn) → e(error)
```

- `Toast.makeText(context, 訊息, 時間).show()`：`this` 是 Context；`.show()` 才真正顯示
- `Log.d("TAG", 訊息)`：第一參數是 TAG（慣例用類別名，方便 Logcat 過濾）
- Android 沒有 `JOptionPane` 那類同步彈窗；真要對話框用 `AlertDialog`（Day 2）

6.1 互動範例：Toast 訊息 + Log 日誌展示 App

功能需求

- 一支 `EditText` 輸入要顯示的訊息
- 「短 Toast」「長 Toast」「置中 Toast」三顆按鈕
- 五顆小按鈕 V D I W E → 對應 Log 五個層級
- 一支 `TextView` 顯示「最後動作」

佈局重點

- 五顆按鈕用水平 `LinearLayout` + `layout_weight="1"` 均分寬度

```
btnLogV.setOnClickListener(v -> {  
    Log.v(TAG, "這是一條 verbose：" + getMessage());  
    showResult("Log.v - verbose（最低層級）已寫入");  
});
```

6.1 程式與語法重點

- Toast 兩種建法：`Toast.makeText(...)` 回傳 `Toast` 物件，最後要 `.show()`
- `toast.setGravity(Gravity.CENTER, 0, 0)`：改變出現位置（X/Y 為偏移量）
- Log 五層級：`v` verbose 最低 → `d` debug → `i` info → `w` warn → `e` error 最高
- Logcat 如何看：Run 之後，下方 Logcat 用左上角層級下拉過濾；搜尋框輸入 `ToastLogDemo`（TAG）

操作	畫面上	Logcat（搜尋 TAG）
短 Toast	下方 ~2 秒	<code>I</code> 一行
中央 Toast	螢幕中央	<code>I</code> （位置=置中）
按 E	顯示已寫入	<code>E</code> 一行（紅色）

AI 動手做 | Toast + Log 展示 App（指南 §5・提示 4）

請幫我寫一個 Android Java App（叫 ToastLogDemo，套件 com.example.toastlog）：

- activity_main.xml：一個 EditText（輸入訊息）、三顆按鈕「短 Toast / 長 Toast / 置中 Toast」、五顆小按鈕 V D I W E（水平平均分寬度）、一個 TextView 顯示最後動作
- MainActivity.java：
 - Toast.LENGTH_SHORT / LENGTH_LONG、setGravity 置中
 - 五顆按鈕分別寫 Log.v/d/i/w/e，TAG 用 "ToastLogDemo"
 - 顯示「我剛做了什麼」到 TextView

請用 lambda，並把 Log 五個層級的意思說明一下

實作後自己在 Logcat 搜尋 `ToastLogDemo` 驗證。

第 7 章 常用 View 元件一覽

元件	XML 標籤	用途	常用屬性
文字	<TextView>	顯示文字	text textSize textColor
輸入框	<EditText>	輸入	hint inputType
按鈕	<Button>	點擊	text
圖片	<ImageView>	顯示圖片	src
勾選	<CheckBox>	複選	checked
單選	<RadioButton>	單選（需 RadioGroup）	checked
下拉	<Spinner>	選單	Adapter 提供

```
<EditText android:layout_width="match_parent" android:layout_height="wrap_content"
    android:hint="輸入數字" android:inputType="number" />
<!-- inputType 常見：text / number / emailAddress / password / phone / numberDecimal -->
```

第 8 章 完整範例一：常用 View 元件互動展示（完整可執行版）

把第 7 章列的常用 View **一次全用上**。這支 App 沒有「過場動畫」，只有滿滿的「不同 View 用不同方法取值」。

功能需求

- `ImageView`：顯示系統內建圖檔，「換圖」按鈕在三張圖間切換
- `Spinner`（下拉）、`Switch`（開關）、`SeekBar`（進度條）、`RatingBar`（星等）
- `Button`「全部更新」：一次把全部狀態寫進一支 `TextView`

怎麼跑： `File → New → New Project → Empty Views Activity`，專案名 `ViewDemo`，Language 選 `Java`，套件 `com.example.viewdemo`。先貼第①步 `activity_main.xml`，再貼第②步 `MainActivity.java`，直接 Run。

關鍵對照：`Adapter` 就是資料與畫面的橋樑，等於 Swing 的 `JList.setModel(...)`。

第 8 章 活動① activity_main.xml（完整檔・1/3）

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="16dp">

    <TextView
        android:id="@+id/tvTitle"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="常用 View 互動展示"
        android:textSize="22sp"
        android:textStyle="bold" />

    <ImageView
        android:id="@+id/imgDemo"
        android:layout_width="120dp"
        android:layout_height="120dp"
        android:src="@android:drawable/ic_menu_gallery"
        android:contentDescription="圖片展示" />

    <Button
        android:id="@+id/btnSwapImage"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="換圖" />
```

第 8 章 活動① activity_main.xml（完整檔・2/3）

```
<Spinner
    android:id="@+id/spinner"
    android:layout_width="match_parent"
    android:layout_height="wrap_content" />

<Switch
    android:id="@+id/switchDemo"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="開關狀態" />

<SeekBar
    android:id="@+id/seekBar"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:max="100"
    android:progress="50" />

<RatingBar
    android:id="@+id/ratingBar"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:numStars="5"
    android:rating="3" />
```

未完，下一頁繼續...

第 8 章 活動① activity_main.xml（完整檔・3/3）

```
<Button
    android:id="@+id/btnUpdateAll"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="全部更新" />

<TextView
    android:id="@+id/tvResult"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="結果："
    android:textSize="16sp" />
</LinearLayout>
```

- `@android:drawable/...`：系統內建資源，免自建檔
- `contentDescription`：無障礙（a11y）用的圖片描述
- `<Spinner>` 需要 `Adapter` 提供選項（`ArrayAdapter`）

第 8 章 活動② MainActivity.java（完整檔・上 1/4）

```
package com.example.viewdemo;

import android.os.Bundle;
import android.view.View;
import android.widget.AdapterView;
import android.widget.AdapterView.OnItemClickListener;
import android.widget.Button;
import android.widget.ImageView;
import android.widget.RatingBar;
import android.widget.SeekBar;
import android.widget.Spinner;
import android.widget.Switch;
import android.widget.TextView;

import androidx.appcompat.app.AppCompatActivity;

import java.util.ArrayList;
import java.util.List;
```

未完，下一頁繼續...

第 8 章 活動② MainActivity.java（完整檔・上 2/4）

```
public class MainActivity extends AppCompatActivity {  
  
    private ImageView imgDemo;  
    private Spinner spinner;  
    private Switch switchDemo;  
    private SeekBar seekBar;  
    private RatingBar ratingBar;  
    private TextView tvResult;  
  
    private int imageIndex = 0;  
    private final int[] images = {  
        android.R.drawable.ic_menu_gallery,  
        android.R.drawable.ic_menu_camera,  
        android.R.drawable.ic_menu_compass  
    };  
};
```

未完，下一頁繼續...

第 8 章 活動② MainActivity.java (完整檔・上 3/4)

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);

    imgDemo    = findViewById(R.id.imgDemo);
    spinner    = findViewById(R.id.spinner);
    switchDemo = findViewById(R.id.switchDemo);
    seekBar    = findViewById(R.id.seekBar);
    ratingBar  = findViewById(R.id.ratingBar);
    tvResult   = findViewById(R.id.tvResult);
    Button btnSwapImage = findViewById(R.id.btnSwapImage);
    Button btnUpdateAll = findViewById(R.id.btnUpdateAll);

    // Spinner : ArrayAdapter 提供選項
    List<String> options = new ArrayList<>();
    options.add("蘋果");
    options.add("香蕉");
    options.add("柳橙");
    ArrayAdapter<String> spinnerAdapter = new ArrayAdapter<>(
        this,
        android.R.layout.simple_spinner_item,
        options);
    spinnerAdapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);
    spinner.setAdapter(spinnerAdapter);
}
```

未完，仍在前一頁 `onCreate` 內。下一頁寫 lambda 監聽器。

第 8 章 活動② MainActivity.java（完整檔・上 4/4）

```
// ✅ 單一方法介面 → lambda
btnSwapImage.setOnClickListener(v -> {
    imageIndex = (imageIndex + 1) % images.length;
    imgDemo.setImageResource(images[imageIndex]);
    tvResult.setText("已換圖：" + (imageIndex + 1) + "/" + images.length);
});

switchDemo.setOnCheckedChangeListener((buttonView, isChecked) ->
    tvResult.setText("開關：已" + (isChecked ? "開啟" : "關閉")));

ratingBar.setOnRatingBarChangeListener((bar, rating, fromUser) ->
    tvResult.setText("星等：" + rating));
```

未完，仍在前一頁 `onCreate` 內。下一頁繼續寫 anonymous class 監聽器。

第 8 章 活動② MainActivity.java (完整檔・下 1/2)

```
// ✗ 多個方法介面 → 只能 anonymous class
spinner.setOnItemSelectedListener(new AdapterView.OnItemSelectedListener() {
    @Override
    public void onItemSelected(AdapterView<?> parent, View view, int position, long id) {
        tvResult.setText("選擇：" + parent.getItemAtPosition(position));
    }

    @Override
    public void onNothingSelected(AdapterView<?> parent) {
    }
});

seekBar.setOnSeekBarChangeListener(new SeekBar.OnSeekBarChangeListener() {
    @Override
    public void onProgressChanged(SeekBar seekBar, int progress, boolean fromUser) {
        tvResult.setText("進度：" + progress + "/" + seekBar.getMax());
    }

    @Override
    public void onStartTrackingTouch(SeekBar seekBar) {
    }

    @Override
    public void onStopTrackingTouch(SeekBar seekBar) {
    }
});
```

第 8 章 活動② MainActivity.java（完整檔・下 2/2）

```
// 全部更新：一次取回全部狀態
btnUpdateAll.setOnClickListener(v -> updateAll());
}

private void updateAll() {
    String fruit = (String) spinner.getSelectedItem(); // Object → 轉型 String
    boolean on = switchDemo.isChecked(); // boolean
    int progress = seekBar.getProgress(); // int
    float rating = ratingBar.getRating(); // float
    tvResult.setText("水果：" + fruit + "\n開關：" + (on ? "開啟" : "關閉")
        + "\n進度：" + progress + "\n星等：" + rating);
}
}
```

- Spinner / SeekBar 監聽器有多個方法 → 只能 anonymous class
- 換圖 / Switch / RatingBar / 按鈕是單一方法介面 → lambda
- `updateAll()` 一次示範四種取值型別

第 8 章 「全部更新」 取值型別

元件	取值方法	回傳型別
Spinner	<code>spinner.getSelectedItem()</code>	<code>Object</code> ，需轉型 <code>(String)</code>
Switch	<code>switchDemo.isChecked()</code>	<code>boolean</code>
SeekBar	<code>seekBar.getProgress()</code>	<code>int</code>
RatingBar	<code>ratingBar.getRating()</code>	<code>float</code> （可含半顆，如 3.5）

判定準則：單一抽象方法介面 → lambda；多方法介面 / 方法覆寫 → 只能寫方法。

AI 動手做 | 常用 View 互動展示（指南 §6-4）

我學到 Android 常用 View，套件 com.example.viewdemo，請幫我產出「常用 View 互動展示」App：

1. activity_main.xml：

- ImageView (120dp，src 用 @android:drawable/ic_menu_gallery) + 「換圖」按鈕
- Spinner (蘋果、香蕉、柳橙) + Switch 「開關狀態」
- SeekBar (max=100，progress=50) + RatingBar (numStars=5，rating=3)
- Button 「全部更新」 + 結果 TextView

2. MainActivity.java：

- Spinner 用 ArrayAdapter，OnItemSelectedListener 用 anonymous class (兩個方法)
- 換圖 (lambda)、Switch 用 OnCheckedChangeListener (lambda)
- SeekBar 用 anonymous class (三個方法)、RatingBar 用 setOnRatingBarChangeListener (lambda)
- 「全部更新」取：spinner.getSelectedItem() (轉 String)、isChecked()、getProgress()、getRating()

請特別標明哪些用 lambda、哪些要用 anonymous class。

驗證：換圖、選單、開關、進度條、星等、全部更新都要即時顯示。

第 9 章 頁面跳轉預告（Day 2）

```
Intent intent = new Intent(this, SecondActivity.class);
startActivity(intent);
```

- `new Intent(出發點, 目的地)`：第一參數 `this`（目前 Activity）、第二參數 `SecondActivity.class`
- `startActivity(...)`：把意圖交給系統，系統開啟新畫面並切換

⚡ `Intent` 類似「開新 JFrame 並 `setVisible(true)`」，但它攜帶「想去哪、帶什麼資料、想做什麼」。

Day 2 會學 `putExtra()` / `getStringExtra()` 在畫面間傳值。

第 10 章 完整範例二：BMI 計算機

佈局重點（activity_main.xml）

- 標題「BMI 計算機」→「身高 (cm)」→ `etHeight`（`inputType="numberDecimal"`）→「體重 (kg)」→ `etWeight` → `btnCalc` 計算 → `tvResult`
- `label TextView + EditText` 成對出現 = 「欄位式」表單排版

程式重點（MainActivity.java）

```
private void calculateBMI() {  
    String heightStr = etHeight.getText().toString().trim();  
    double heightCm = Double.parseDouble(heightStr); // 空值檢查 → Toast + return  
    double bmi = weightKg / ((heightCm / 100.0) * (heightCm / 100.0));  
    // 分類：<18.5 過輕、<24 正常、<27 過重、否則肥胖  
    String result = String.format("BMI = %.1f\n分類：%s", bmi, category);  
    tvResult.setText(result);  
}
```

驗證：輸入 170 / 60 → 「BMI = 20.8 分類：正常」

AI 動手做 | BMI 計算機（指南 §6-1）

我是 Java 開發者（熟悉 Swing），正在學 Android（Java + XML）。

請幫我完成「BMI 計算機」App，套件名 `com.example.bmicalc`：

1. `activity_main.xml`：

- 標題「BMI 計算機」28sp 粗體
- 兩個 `EditText`：身高 (cm) 用 `inputType="numberDecimal"`，體重 (kg) 用 `numberDecimal`
- 一個按鈕「計算 BMI」、一個結果 `TextView`

2. `MainActivity.java`：

- `findViewById` 綁定；點擊按鈕用 `lambda` 呼叫 `calculateBMI()`
- 空值或 `<=0` 時用 `Toast` 提示
- $BMI = \text{體重} / (\text{身高m})^2$ ，分類：`<18.5` 過輕、`<24` 正常、`<27` 過重、否則肥胖
- `String.format` 顯示 BMI 一位小數 + 分類

請用 `lambda` 寫法。

練習擴充：幫 BMI 加上「Reset 清空」按鈕並顯示「標準體重範圍」。

驗證：170 / 60 → 「BMI = 20.8 分類：正常」

第 11 章 完整範例三：溫度轉換器

需求：輸入溫度；RadioButton 選「C→F」或「F→C」；顯示 2 位小數；空白用 Toast。

佈局重點

- `<RadioGroup>` 是容器，包住 `<RadioButton>`；同組內「互相排斥」
- `android:checked="true"`：預設選中「攝氏→華氏」
- `inputType="numberDecimal|numberSigned"`：允許小數 + 負號（可輸 -5.5）

```
private void convert() {  
    if (TextUtils.isEmpty(input)) { Toast...; return; } // 連 null 一起擋掉  
    double value = Double.parseDouble(input);  
    int checkedId = radioGroup.getCheckedRadioButtonId();  
    if (checkedId == R.id.rbToF) {  
        result = value * 9.0 / 5.0 + 32; // C -> F  
    } else {  
        result = (value - 32) * 5.0 / 9.0; // F -> C  
    }  
    tvResult.setText(String.format("結果：%.2f", result));  
}
```

驗證：100/C→F → 212.00；32/F→C → 0.00；-40/C→F → -40.00

AI 動手做 | 溫度轉換器（指南 §6-2）

我學到「溫度轉換器」，套件 `com.example.tempconvert`，請幫我產出：

1. `activity_main.xml`：
 - 標題「溫度轉換器」
 - `EditText` (`etTemp`)，`inputType="numberDecimal|numberSigned"`
 - `RadioGroup` 水平，兩個 `RadioButton`：「攝氏→華氏」預設選中、「華氏→攝氏」
 - 轉換按鈕、結果 `TextView`
2. `MainActivity.java`：
 - 用 `TextUtils.isEmpty` 檢查空值（`Toast` 提示）
 - `radioGroup.getCheckedRadioButtonId()` 判斷方向
 - $C \rightarrow F: value * 9/5 + 32$ ； $F \rightarrow C: (value - 32) * 5/9$
 - `String.format` 顯示兩位小數

練習擴充：把 `RadioButton` 改成 `Spinner` 下拉選方向、兩個刻度都顯示。

驗證： $100/C \rightarrow F \rightarrow 212.00$ ； $32/F \rightarrow C \rightarrow 0.00$ 。

第 12 章 完整範例四：登入表單

需求：帳號、密碼欄；CheckBox「顯示密碼」；判斷 `admin / 1234`；成功 Toast、失敗 AlertDialog。

佈局重點

- `android:inputType="textPassword"`：密碼欄關鍵屬性 → 內容以 ●●● 遮蔽
- `cbShow` 勾選與否的明文/隱藏切換邏輯在 Java

```
cbShow.setOnCheckedChangeListener((buttonView, isChecked) -> {  
    int type = isChecked  
        ? android.text.InputType.TYPE_CLASS_TEXT           // 明文  
        : android.text.InputType.TYPE_CLASS_TEXT  
            | android.text.InputType.TYPE_TEXT_VARIATION_PASSWORD; // 遮蔽  
    etPassword.setInputType(type);  
});
```

第 12 章 登入驗證與 AlertDialog

```
private void login() {  
    String account = etAccount.getText().toString().trim(); // 帳號去空白  
    String password = etPassword.getText().toString();      // 密碼保留原樣  
    if (account.equals("admin") && password.equals("1234")) {  
        tvMessage.setText("歡迎登入：" + account);  
        Toast.makeText(this, "登入成功", Toast.LENGTH_SHORT).show();  
    } else {  
        new AlertDialog.Builder(this) // 取代 JOptionPane 彈窗  
            .setTitle("登入失敗")  
            .setMessage("帳號或密碼錯誤")  
            .setPositiveButton("確定", null) // null = 按確定只關窗  
            .show();  
    }  
}
```

- OnCheckedChangeListener 單一方法 → lambda
- 帳號 trim()、密碼**不去空白**（保持精確比對）

AI 動手做 | 登入表單（指南 §6-3）

我學到「登入表單」，套件 `com.example.loginform`，請幫我產出：

1. `activity_main.xml`：
 - 標題「會員登入」
 - EditText 帳號 (`inputType="text"`)、EditText 密碼 (`inputType="textPassword"`)
 - CheckBox「顯示密碼」、登入按鈕、狀態 TextView (初始「尚未登入」)
2. `MainActivity.java`：
 - CheckBox 勾選時用 `setInputType` 切換密碼明文/遮蔽
 - 登入判斷帳密是否 `admin / 1234`，成功 Toast「登入成功」+ 歡迎詞
 - 失敗用 `AlertDialog` 彈出「帳號或密碼錯誤」
 - 點擊用 `lambda ; OnCheckedChangeListener` 用 `lambda`

AI 動手做 | 擴充與解釋（指南 §8）

擴充自己會的功能

我 Day 1 已會用 `findViewById + lambda + Toast`。
請教我「怎麼」把 BMI 結果在按下按鈕時也彈出 `AlertDialog` 顯示，並說明每一行在做什麼。

對照第 13 章測驗，把 AI 當成互動版。

第 13 章 自我測驗（先寫再對答案）

1. `layout_width="match_parent"` 和 `"wrap_content"` 分別是什麼意思？
2. `findViewById(R.id.btnShow)` 的 `R.id.btnShow` 指的是什麼？
3. 文字為何建議用 `sp` 而不是 `px`？
4. `onCreate` 何時被呼叫？裡面一定要做什麼？
5. `Toast` 和 `JOptionPane` 的主要差異？
6. 什麼情況下可以用 `lambda`？有哪些限制？
7. `Switch`、`SeekBar`、`RatingBar` 取值方法各回傳什麼型別？為何用各自的方法？

測驗解答（1-4）

1. match_parent / wrap_content

- `match_parent`：填滿父容器（≈ Fill）；`wrap_content`：只包住內容（≈ Preferred Size）

2. R.id.btnShow

- `R` 是編譯時自動產生的資源類別；`R.id.btnShow` 是 `@+id/btnShow` 對應的唯一整數 ID；`findViewById` 透過它找到 View

3. 為何用 sp 不是 px

- `sp` 依系統字型設定縮放；`px` 固定、不同密度螢幕大小不一。排版用 `dp`、文字用 `sp`

4. onCreate

- Activity 首次建立時第一個被呼叫的生命週期方法；一定要 `super.onCreate(...)` + 至少一次 `setContentView(...)`

測驗解答（5–7）

5. Toast vs JOptionPane

- Toast 非同步、不阻塞、自動消失、無法取回饋；JOptionPane 同步、會阻塞、可取回傳值。Android 用 `AlertDialog` 才類似彈窗

6. 何時可用 lambda

- 只有「單一抽象方法」介面（functional interface）才能用 lambda；多方法介面、方法覆寫（如 `onActivityResult`）不行；需 Java 8+

7. Switch / SeekBar / RatingBar 取值

- `isChecked()` → `boolean`；`getProgress()` → `int`；`getRating()` → `float`（可半顆，如 3.5）
- 各元件狀態型別天生不同，因此各自有對應的取值方法

AI 動手做 | 建議複習流程（把 AI 變成互動練習）

1. 先自己照教材做一遍（不看 AI）
2. 再用上面的提示讓 AI 產生同一支程式
3. 比對你寫的 vs AI 寫的，看哪裡不同、為什麼
4. 讓 AI 出測驗，驗證你真的懂
5. 用「練習擴充提示」新增功能，挑戰自己

這樣做，AI 不是幫你抄作業，而是幫你「反覆練習 + 理解差異」，這是最有效的學法。

